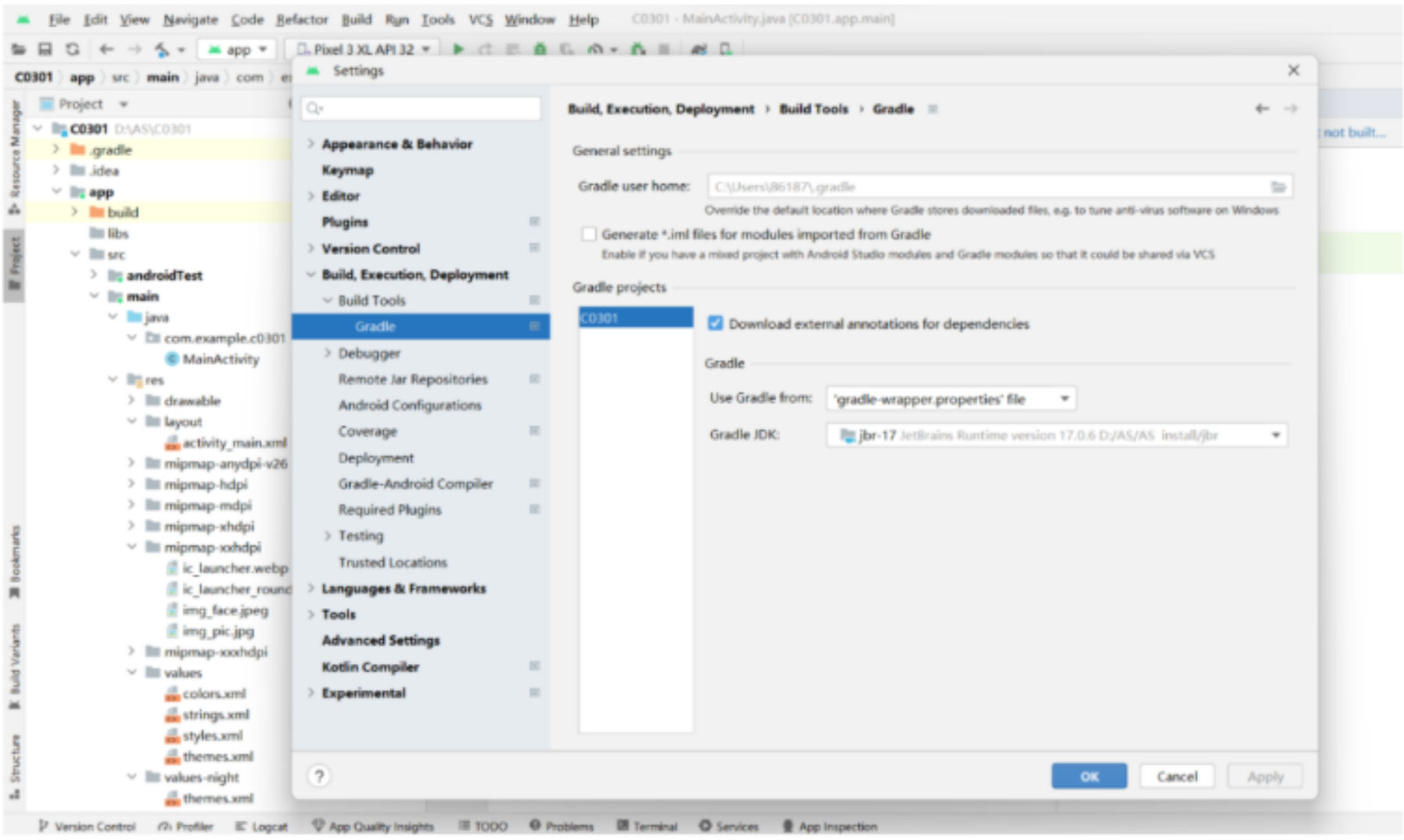
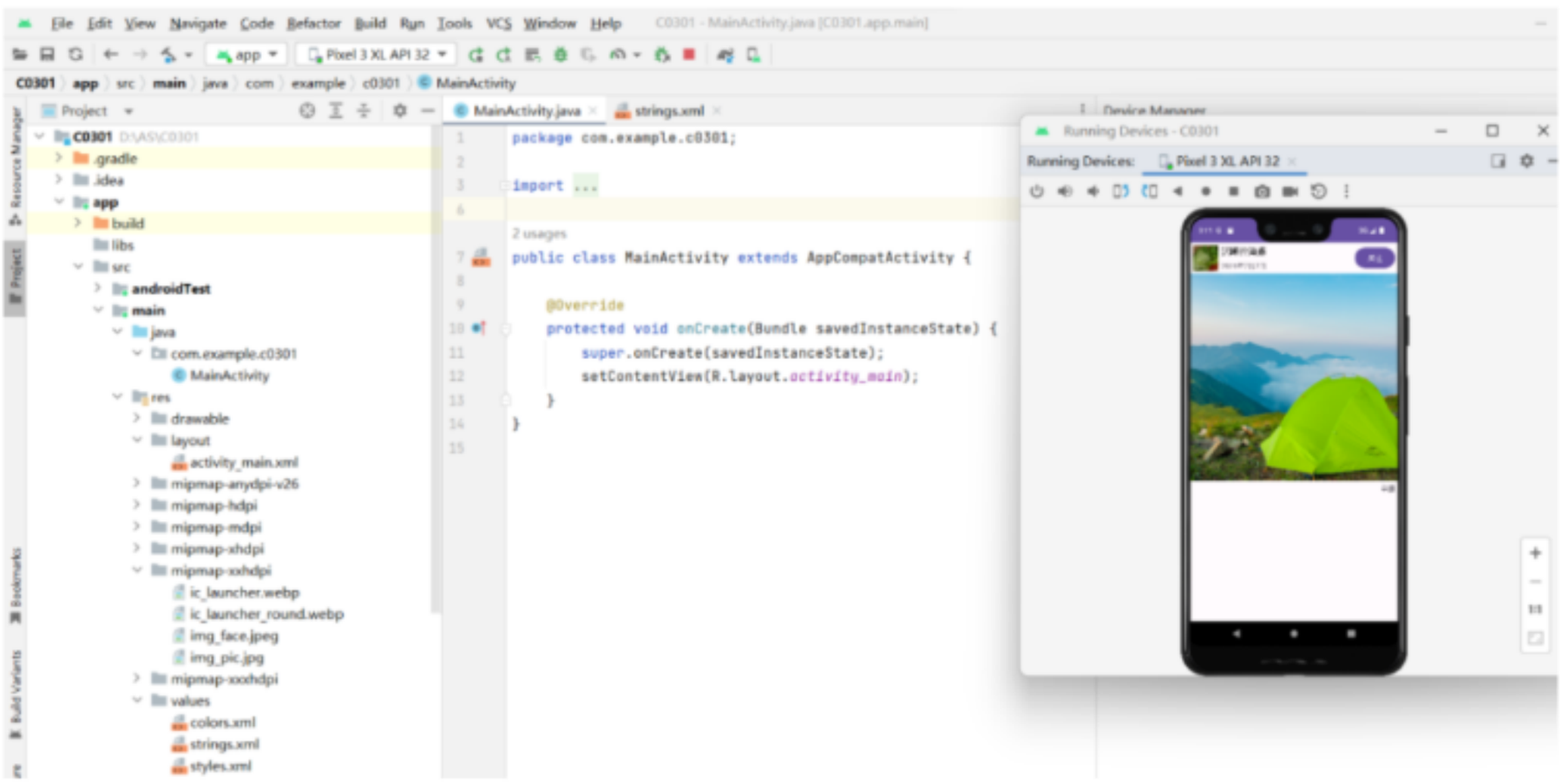
第一章作业：

P22 1. 下载最新版本的 Android Studio，安装后如果需要更新 Gradle，则将其更新到最新版本。



P22 2. 创建一个任意类型的手机 App 工程，分别使用虚拟设备和物理设备运行。



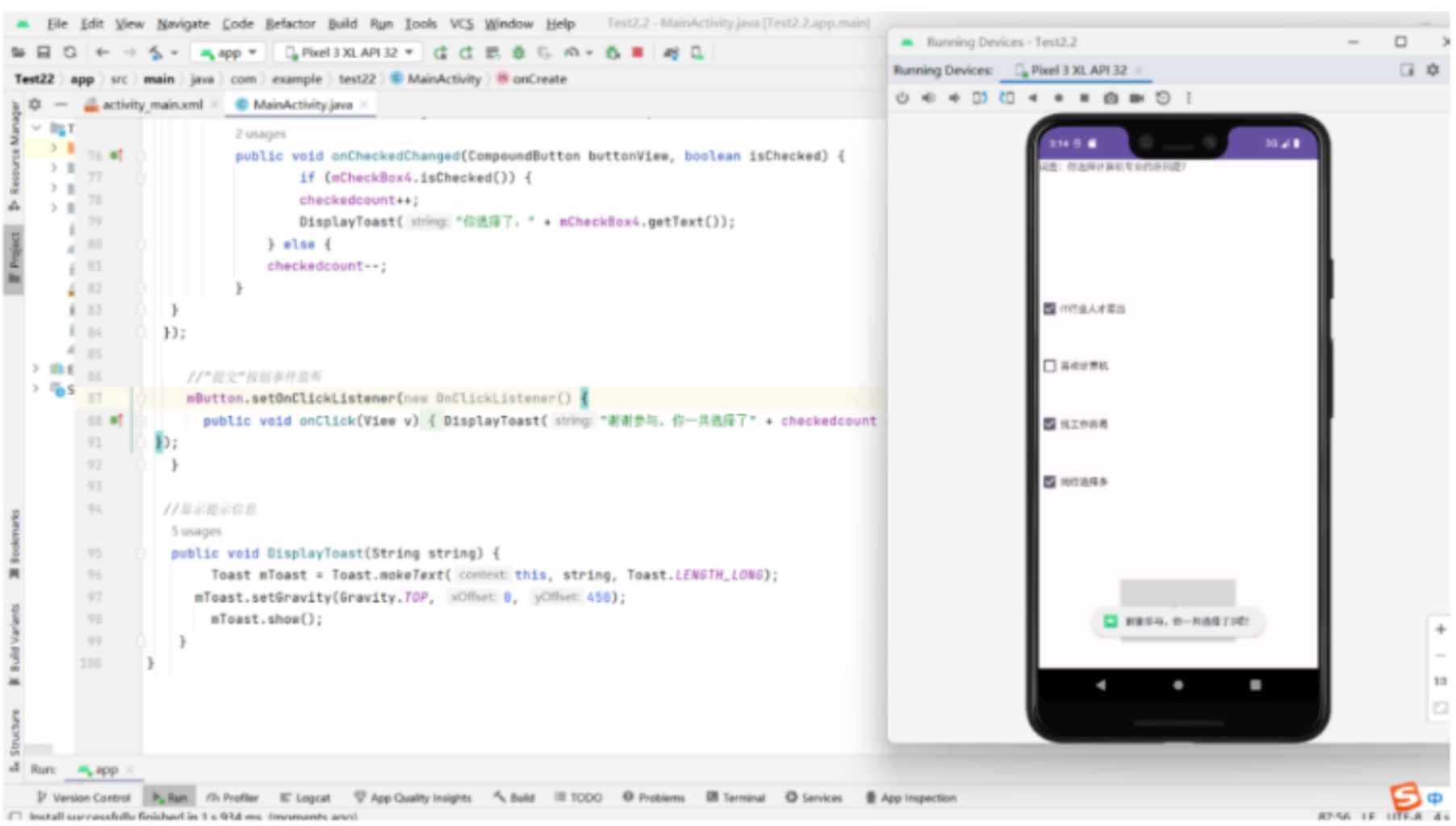
第二章作业：

P73 2.2 实现多选题的界面效果。

```
public class MainActivity extends Activity {
 TextView mTextView;                          //TextView 对象，用于显示问题
   CheckBox mCheckBox1;                       //CheckBox 对象，用于显示选项 1
      CheckBox mCheckBox2;                    //CheckBox 对象，用于显示选项 2
    CheckBox mCheckBox3;                      //CheckBox 对象，用于显示选项 3
    CheckBox mCheckBox4;                      //CheckBox 对象，用于显示选项 4
```

```
Button mButton;                                          //Button 对象，用于显示提交按钮

   int checkedcount = 0;                                 //计数器，用于统计选中的个数
   @Override
  public void onCreate(Bundle savedInstanceState) {
              super.onCreate(savedInstanceState);
                setContentView(R.layout.activity_main);//加载布局文件

            //加载控件
         mTextView = (TextView)this.findViewById(R.id.textview);
          mCheckBox1 = (CheckBox)this.findViewById(R.id.checkbox1);
           mCheckBox2 = (CheckBox)this.findViewById(R.id.checkbox2);
          mCheckBox3 = (CheckBox)this.findViewById(R.id.checkbox3);
          mCheckBox4 = (CheckBox)this.findViewById(R.id.checkbox4);
           mButton = (Button)this.findViewById(R.id.button);

           //选项 1 事件监听
           mCheckBox1.setOnCheckedChangeListener(new
OnCheckedChangeListener() {
                  public     void     onCheckedChanged(CompoundButton     buttonView,
boolean isChecked) {
                    if (mCheckBox1.isChecked()) {
                          checkedcount++;
                          DisplayToast("你选择了：" + mCheckBox1.getText());
                    } else {
                        checkedcount--;
                  }
              }
          });

          //选项 2 事件监听
          mCheckBox2.setOnCheckedChangeListener(new
OnCheckedChangeListener() {
           public  void  onCheckedChanged(CompoundButton  buttonView,  boolean
isChecked) {
                if (mCheckBox2.isChecked()) {
                          checkedcount++;
                            DisplayToast("你选择了：" + mCheckBox2.getText());
                           } else {
                                          checkedcount--;
                                }
                }
          });

              //选项 3 事件监听
          mCheckBox3.setOnCheckedChangeListener(new
OnCheckedChangeListener() {
                public  void  onCheckedChanged(CompoundButton  buttonView,  boolean
isChecked) {
                     if (mCheckBox3.isChecked()) {
                          checkedcount++;
                          DisplayToast("你选择了：" + mCheckBox3.getText());
                    } else {
                        checkedcount--;
                    }
              }
         });
```

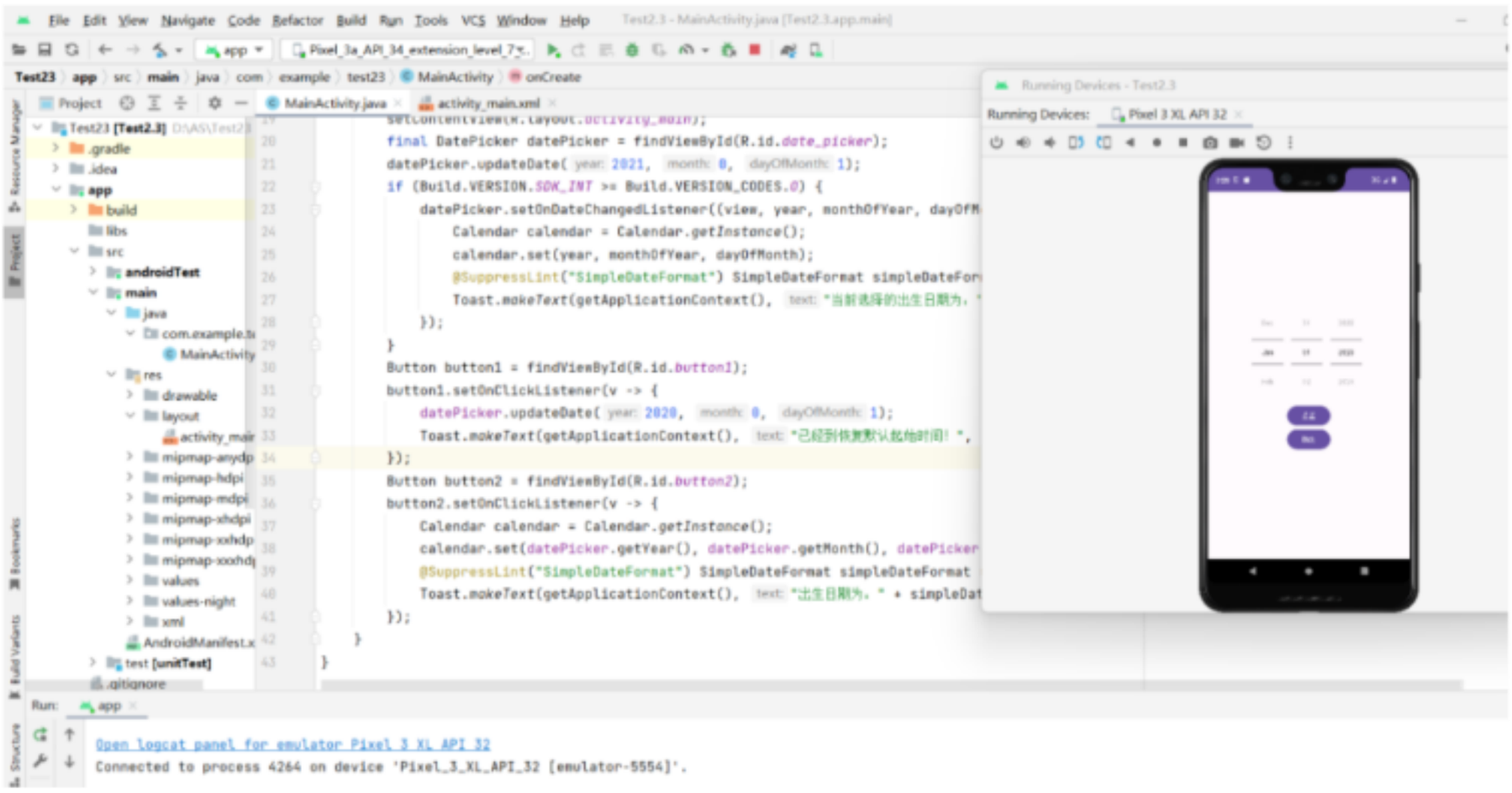
```
        //选项 4 事件监听
         mCheckBox4.setOnCheckedChangeListener(new
OnCheckedChangeListener() {
              public void onCheckedChanged(CompoundButton buttonView, boolean
isChecked) {
                    if (mCheckBox4.isChecked()) {
                    checkedcount++;
                    DisplayToast("你选择了：" + mCheckBox4.getText());
                } else {
                checkedcount--;
            }
        }
    });

        //” 提交 “按钮事件监听
        mButton.setOnClickListener(new OnClickListener() {
           public void onClick(View v) {
                    DisplayToast("谢谢参与，你一共选择了" + checkedcount + "项！ ");
            }
});
    }

   //显示提示信息
    public void DisplayToast(String string) {
          Toast mToast = Toast.makeText(this, string, Toast.LENGTH_LONG);
       mToast.setGravity(Gravity.TOP, 0, 450);
          mToast.show();
     }
}
```



p73 2.3 实现选择出生日期的界面效果。

```
public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        final DatePicker datePicker = findViewById(R.id.date_picker);
        datePicker.updateDate(2021, 0, 1);
```

```
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            datePicker.setOnDateChangedListener((view, year, monthOfYear, dayOfMonth)
-> {
                Calendar calendar = Calendar.getInstance();
                calendar.set(year, monthOfYear, dayOfMonth);
                @SuppressLint("SimpleDateFormat") SimpleDateFormat simpleDateFormat
= new SimpleDateFormat("yyyy 年 MM 月 dd 日");
                Toast.makeText(getApplicationContext(), "当前选择的出生日期为：" +
simpleDateFormat.format(calendar.getTime()), Toast.LENGTH_SHORT).show();
            });
        }
        Button button1 = findViewById(R.id.button1);
        button1.setOnClickListener(v -> {
            datePicker.updateDate(2020, 0, 1);
            Toast.makeText(getApplicationContext(), "已经到恢复默认起始时间！",
Toast.LENGTH_LONG).show();
        });
        Button button2 = findViewById(R.id.button2);
        button2.setOnClickListener(v -> {
            Calendar calendar = Calendar.getInstance();
            calendar.set(datePicker.getYear(), datePicker.getMonth(),
datePicker.getDayOfMonth());
            @SuppressLint("SimpleDateFormat") SimpleDateFormat simpleDateFormat =
new SimpleDateFormat("yyyy 年 MM 月 dd 日");
            Toast.makeText(getApplicationContext(), "出生日期为：" +
simpleDateFormat.format(calendar.getTime()), Toast.LENGTH_SHORT).show();
        });
    }
}
```

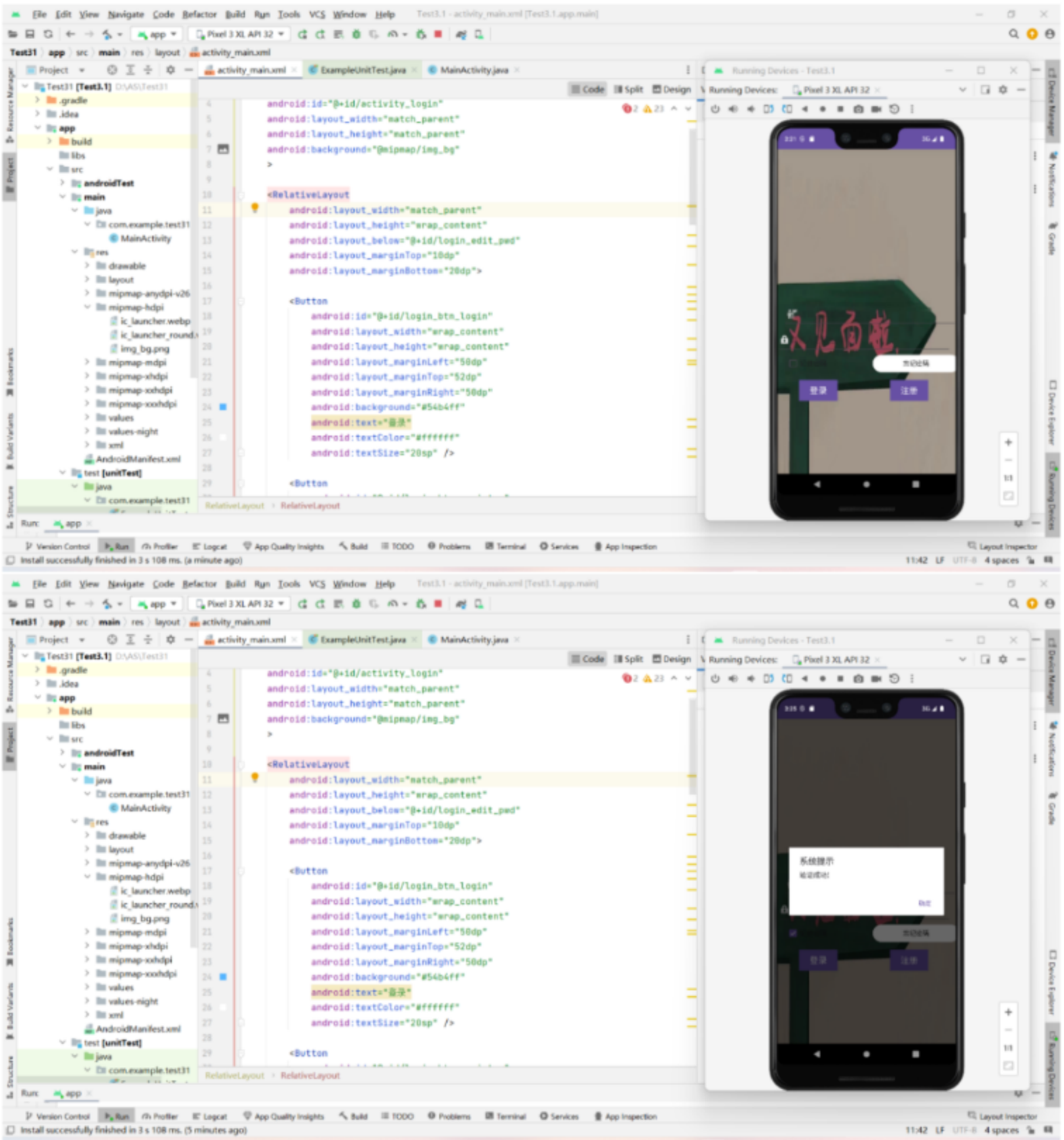


第三章作业：

实现注册界面的布局效果，至少包括用户名、密码、忘记密码、登录等控件。

```
public class MainActivity extends AppCompatActivity{
    @Override
```

```
    protected void onCreate(Bundle savedInstanceState){
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Button button=(Button)findViewById(R.id.login_btn_forgetregister);
        button.setOnClickListener(new View.OnClickListener(){
            @Override
            public void onClick(View v){new AlertDialog.Builder(MainActivity.this).setTitle("
系统提示").setMessage("请输入验证信息进行验证!")
                    .setPositiveButton("确定",new DialogInterface.OnClickListener(){
                        @Override
                        public void onClick(DialogInterface dialog,int which){
                            finish();
                        }
                    }).setNegativeButton("返回",new DialogInterface.OnClickListener(){
                        @Override
                        public void onClick(DialogInterface dialog,int which){
                        }

                    }).show();
            }
        });
        Button button1=(Button)findViewById(R.id.login_btn_login);
        button1.setOnClickListener(new View.OnClickListener(){
            @Override
            public void onClick(View v){
                new AlertDialog.Builder(MainActivity.this).setTitle("系统提示").setMessage("
验证成功!")
                        .setNegativeButton("确定",new DialogInterface.OnClickListener(){
                            @Override
                            public void onClick(DialogInterface dialog,int which){
                            }
                        }).show();
            }
        });
        Button button2=(Button)findViewById(R.id.login_btn_register);
        button2.setOnClickListener(new View.OnClickListener(){
            @Override
            public void onClick(View v){
                new AlertDialog.Builder(MainActivity.this).setTitle("系统提示").setMessage("
注册成功!")
                        .setNegativeButton("确定",new DialogInterface.OnClickListener(){
                            @Override
                            public void onClick(DialogInterface dialog,int which){
                            }
                        }).show();
            }
        });
    }
}
```



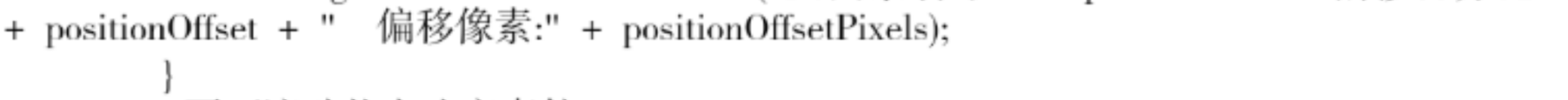
第四章作业：

p122 4.3 实现京东首页的轮播广告效果，至少包括 3 个产品广告的图像。

```
public class MainActivity extends AppCompatActivity implements View.OnClickListener {
    private ViewPager mViewPager;
    private TextView mPagePositionTextView;
    private TextView mPageStateTextView;
    private ImageView mPage0;
    private ImageView mPage1;
    private ImageView mPage2;
    private ImageView mPage3;
    private int mCurrentPosition;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        mPagePositionTextView = findViewById(R.id.text_view_page_position);
```

```
mPageStateTextView = findViewById(R.id.text_view_page_state);
mPage0 = findViewById(R.id.image_view_page0);
mPage1 = findViewById(R.id.image_view_page1);
mPage2 = findViewById(R.id.image_view_page2);
mPage3 = findViewById(R.id.image_view_page3);
mPage0.setOnClickListener(this);
mPage1.setOnClickListener(this);
mPage2.setOnClickListener(this);
mPage3.setOnClickListener(this);
ImageView leftImageView = findViewById(R.id.image_button_left);
ImageView rightImageView = findViewById(R.id.image_button_right);
        leftImageView.setOnClickListener(this);
        rightImageView.setOnClickListener(this);
        mViewPager = findViewById(R.id.view_pager);
        mViewPager.addOnPageChangeListener(new PageChangeListener());
        init();
    }
    // 初始化
    private void init(){
        ArrayList<View> pagerViews = new ArrayList<>();
        pagerViews.add(View.inflate(this, R.layout.view_pager_welcome0, null));
        pagerViews.add(View.inflate(this, R.layout.view_pager_welcome1, null));
        pagerViews.add(View.inflate(this, R.layout.view_pager_welcome2, null));
        pagerViews.add(View.inflate(this, R.layout.view_pager_welcome3, null));
        WelcomePagerAdapter welcomePagerAdapter = new
WelcomePagerAdapter(pagerViews);
        mViewPager.setAdapter(welcomePagerAdapter);
    }
    //单击监听事件
@Override
public void onClick(View v) {
    int id = v.getId();
    if (id == R.id.image_button_left) {
        if (mCurrentPosition > 0) {
            mCurrentPosition--;
        }
        mViewPager.setCurrentItem(mCurrentPosition, true);
    } else if (id == R.id.image_button_right) {
        if (mCurrentPosition < 3) {
            mCurrentPosition++;
        }
        mViewPager.setCurrentItem(mCurrentPosition, true);
    } else if (id == R.id.image_view_page0) {
        mCurrentPosition = 0;
        mViewPager.setCurrentItem(mCurrentPosition, true);
    } else if (id == R.id.image_view_page1) {
        mCurrentPosition = 1;
        mViewPager.setCurrentItem(mCurrentPosition, true);
    } else if (id == R.id.image_view_page2) {
        mCurrentPosition = 2;
        mViewPager.setCurrentItem(mCurrentPosition, true);
    } else if (id == R.id.image_view_page3) {
        mCurrentPosition = 3;
        mViewPager.setCurrentItem(mCurrentPosition, true);
    }
}
//页面改变监听器
public class PageChangeListener implements ViewPager.OnPageChangeListener {
```

```
//页面选择事件
public void onPageSelected(int position) {
    //翻页时当前 page，改变当前状态圆点图片。
    mCurrentPosition = position;
    switch (position) {
        case 0:
            mPage0.setImageResource(R.mipmap.img_page_now);
            mPage1.setImageResource(R.mipmap.img_page);
            mPage2.setImageResource(R.mipmap.img_page);
            mPage3.setImageResource(R.mipmap.img_page);
            break;
        case 1:
            mPage1.setImageResource(R.mipmap.img_page_now);
            mPage0.setImageResource(R.mipmap.img_page);
            mPage2.setImageResource(R.mipmap.img_page);
            mPage3.setImageResource(R.mipmap.img_page);
            break;
    case 2:
            mPage2.setImageResource(R.mipmap.img_page_now);
            mPage0.setImageResource(R.mipmap.img_page);
            mPage1.setImageResource(R.mipmap.img_page);
            mPage3.setImageResource(R.mipmap.img_page);
            break;
    case 3:
            mPage3.setImageResource(R.mipmap.img_page_now);
            mPage0.setImageResource(R.mipmap.img_page);
            mPage1.setImageResource(R.mipmap.img_page);
            mPage2.setImageResource(R.mipmap.img_page);
            break;
            }
        }
        //页面滚动事件
        @SuppressLint("SetTextI18n")
        public void onPageScrolled(int position, float positionOffset, int positionOffsetPixels)
{
            //mPagePositionTextView.setText("页面索引号:" + position + "  偏移百分比:"
+ positionOffset + "  偏移像素:" + positionOffsetPixels);
        }
        //页面滚动状态改变事件
        public void onPageScrollStateChanged(int state) {
            switch (state) {
                case ViewPager.SCROLL_STATE_IDLE:
                    mPageStateTextView.setText("空闲状态");
                    break;
                case ViewPager.SCROLL_STATE_DRAGGING:
                    mPageStateTextView.setText("拖动状态");
                    break;
                case ViewPager.SCROLL_STATE_SETTLING:
                    mPageStateTextView.setText("结束状态");
                    break;
            }
        }
    }
```



第五章：

p150 5.1 实现登录后显示用户名的效果。输入用户名、密码后登录，启动一个新的

Activity
显示用户名。
MainActivity 代码：

```
public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

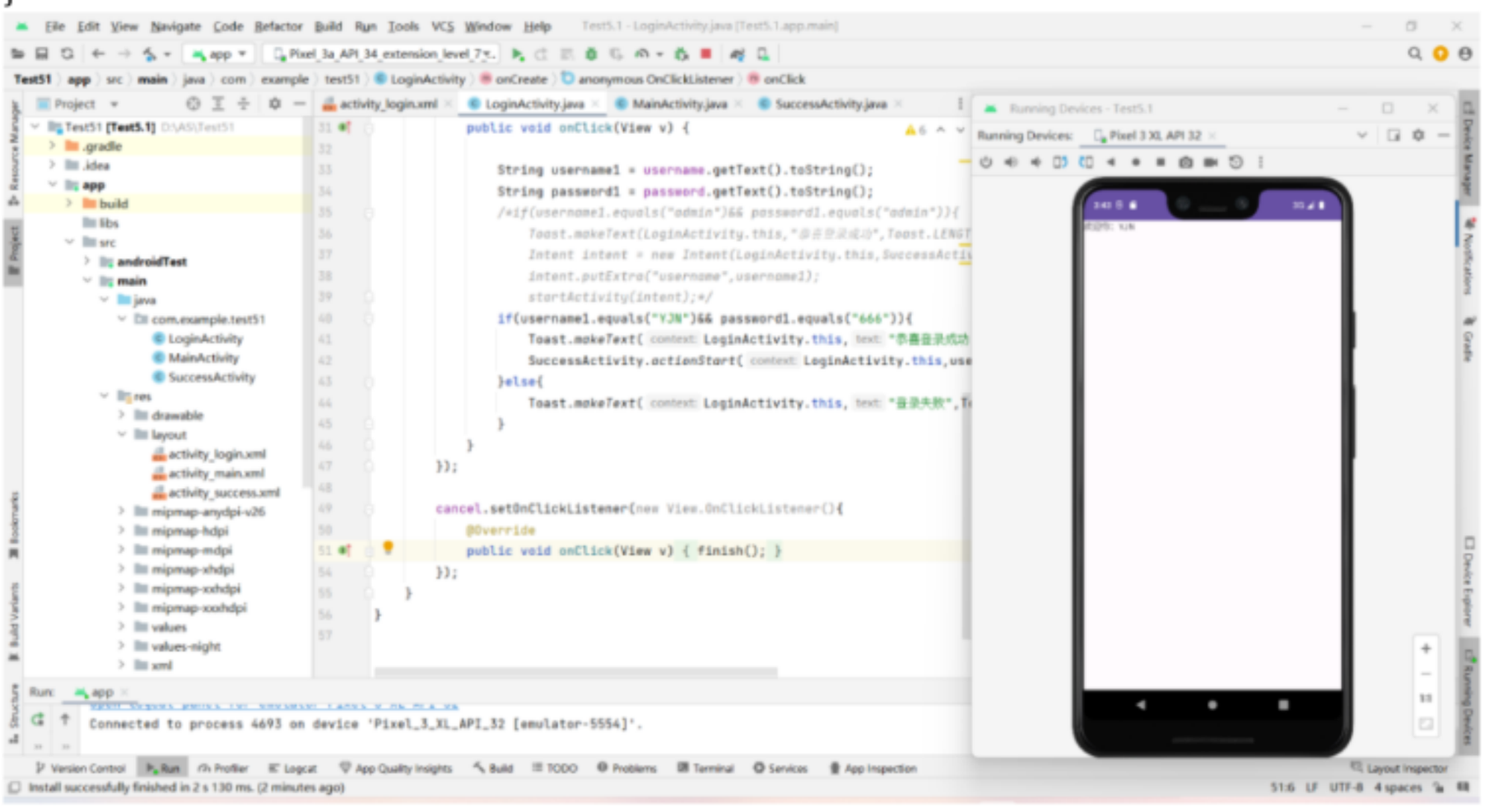
SuccessActivity 代码：

```
public class SuccessActivity extends AppCompatActivity {


    public static void actionStart(Context context,String username1){
        Intent intent = new Intent(context, SuccessActivity.class);
        intent.putExtra("username",username1);
        context.startActivity(intent);
    }

    @SuppressLint("SetTextI18n")
    @Override
    protected void onCreate(Bundle savedInstanceState) {

        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_success);

            TextView textView=(TextView) findViewById(R.id.textView);

            Intent intent = getIntent();
            String username = intent.getStringExtra("username");
            textView.setText("欢迎你：  "+username);
        }
}
```

LoginActivity 代码：

```
public class LoginActivity extends AppCompatActivity {
    private EditText username;
    private EditText password;
    private Button    login;
    private Button cancel;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_login);

username = (EditText) findViewById(R.id.username);
password = (EditText) findViewById(R.id.password);
login = (Button) findViewById(R.id.btn_login);
cancel = (Button) findViewById(R.id.btn_cancel);

login.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {

        String username1 = username.getText().toString();
```

```
                String password1 = password.getText().toString();
                /*if(username1.equals("admin")&& password1.equals("admin")){
                    Toast.makeText(LoginActivity.this,"恭喜登录成功
",Toast.LENGTH_SHORT).show();
                    Intent intent = new
Intent(LoginActivity.this,SuccessActivity.class);
                    intent.putExtra("username",username1);
                    startActivity(intent);*/
                if(username1.equals("YJN")&& password1.equals("666")){
                    Toast.makeText(LoginActivity.this,"恭喜登录成功
",Toast.LENGTH_SHORT).show();
                    SuccessActivity.actionStart(LoginActivity.this,username1);
                }else{
                    Toast.makeText(LoginActivity.this,"登录失败
",Toast.LENGTH_SHORT).show();
                }
            }
        });

        cancel.setOnClickListener(new View.OnClickListener(){
            @Override
            public void onClick(View v) {
                finish();
            }
        });
    }
}
```



第六章：

p176 6.3 通过广播接收器实现两个 App 之间通过自定义广播进行数据通信。

使用 Android Studio 提供的快捷方式来创建一个广播接收器来准备接收此广播，点击 app —>New—>Other—>Broadcast Receiver

收到自定义广播时会弹出“received in MyBroadcastReceiver”的提示

MyReceiver1.java 代码：

```
public class MyReceiver1 extends BroadcastReceiver {
```

```
        private static final String TAG = "MyBroadcastReceiver";
        @Override
        public void onReceive(Context context, Intent intent) {
            Log.d(TAG, "-----------Receivers1-----------");
            Log.d(TAG, "Action: " + intent.getAction());
            Log.d(TAG, "URI: " + intent.toUri(Intent.URI_INTENT_SCHEME));
            Log.d(TAG, "自定义广播的 info：" + intent.getStringExtra("info"));
        }
}
```

AppCompatActivity.java 代码：

```
public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        //发送标准广播
        findViewById(R.id.button).setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                Intent intent=new Intent();
                intent.setAction("MyBroadcastReceiver.Custom");
                intent.putExtra("info","广播来了！ ");
                intent.addFlags(Intent.FLAG_ACTIVITY_PREVIOUS_IS_TOP);
                sendBroadcast(intent);
            }
        });
    }
}
```

MyReceiver2 .java 代码:

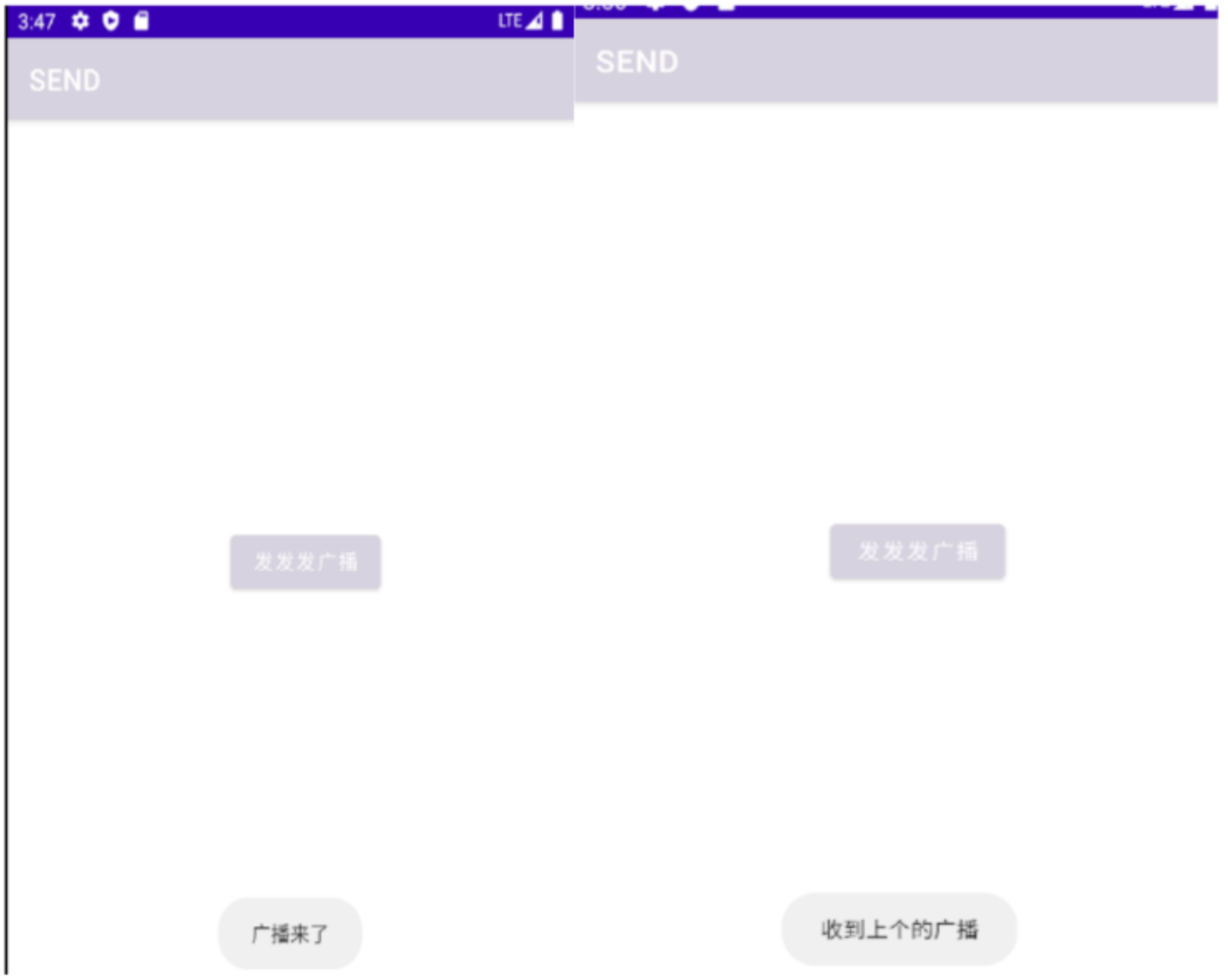
```
public class MyReceiver2 extends BroadcastReceiver {
    private static final String TAG = "MyBroadcastReceiver";
    @Override
    public void onReceive(Context context, Intent intent) {
        // TODO: This method is called when the BroadcastReceiver is receiving
        // an Intent broadcast.
        Log.d(TAG, "-----------Receiver-----------");
        Log.d(TAG, "Action: " + intent.getAction());
        Log.d(TAG, "URI: " + intent.toUri(Intent.URI_INTENT_SCHEME));
        //  自定义广播
        if ("MyBroadcastReceiver.Custom".equals(intent.getAction())) {
            Log.d(TAG, "自定义广播的 info：" + intent.getStringExtra("info"));
        }
    }
}
```

AppCompatActivity.java

```
public class MainActivity extends AppCompatActivity {
    private MyReceiver2 myReceiver = new MyReceiver2();
    private boolean mRegistered = false;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);//注册广播接收器
        IntentFilter intentFilter = new IntentFilter();
        intentFilter.addAction(ConnectivityManager.CONNECTIVITY_ACTION);
        intentFilter.addAction(Intent.ACTION_SCREEN_ON);
```

```
                intentFilter.addAction(Intent.ACTION_SCREEN_OFF);
                intentFilter.addAction(Intent.ACTION_USER_PRESENT);
                intentFilter.addAction(Intent.ACTION_BATTERY_OKAY);
                intentFilter.addAction(Intent.ACTION_BATTERY_LOW);
                intentFilter.addAction(Intent.ACTION_BATTERY_CHANGED);
//                      intentFilter.addAction("com.vt.c0701.MyBroadcastReceiver.ss");
                registerReceiver(myReceiver, intentFilter);
                mRegistered = true;
        }
        @Override
        protected void onDestroy() {
                super.onDestroy();
                if(mRegistered) {
                        unregisterReceiver(myReceiver);
                }
        }
}
```

运行虚拟机点击发送广播 button 若收到广播即可看到 Toast 提示
如左图：
更改后运行虚拟机，点击 button 弹出消息如图
如右图



第七章作业：

p213 7.2 使用 SQLite 实现记录登录时间的功能，并且能够查询登录的历史记录。

SQLiteDBHelper.java 代码：

```
public class SQLiteDBHelper extends SQLiteOpenHelper {

    // 创建数据库
    static final String CREATE_SQL[] = {
            "CREATE TABLE news (" +
                    "_id    INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT," +
                    "title    varchar," +
                    "content    varchar," +
                    "keyword    varchar," +
                    "category    varchar," +
                    "author    INTEGER," +
                    "publish_time    varchar" +
                    ")",
            "CREATE TABLE user (" +
                    "_id    INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT," +
                    "name    varchar," +
                    "password    varchar," +
                    "email    varchar," +
                    "phone    varchar," +
                    "address    varcher" +
                    ")",
            "INSERT INTO user VALUES (1,'admin',123,'admin@163.com',123,'洛阳')",
            "INSERT INTO user VALUES (2,'zhangsan',123,'zhangsan@163.com',123,'北京')",
            "INSERT INTO user VALUES (3,'lisi',123,'lisi@163.com',123,'上海')",
            "INSERT INTO user VALUES (4,'wangwu',123,'wangwu@163.com',123,'深圳')"
    };

    // 调用父类的构造方法(便于之后进行初始化赋值)
    public SQLiteDBHelper(@Nullable Context context, @Nullable String name, int version) {
        super(context, name, null, version);
    }

    /**
     * Called when the database is created for the first time. This is where the
     * creation of tables and the initial population of the tables should happen.
     *
     * @param db The database.
     */
    @Override
    public void onCreate(SQLiteDatabase db) {

        for (int i = 0; i < CREATE_SQL.length; i++) {
            db.execSQL(CREATE_SQL[i]);
        }

        Log.i("sqlite_____", "Finished Database");

    }

    @Override
```

```
        public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {

        }
}
```

SQLiteListActivity 代码：

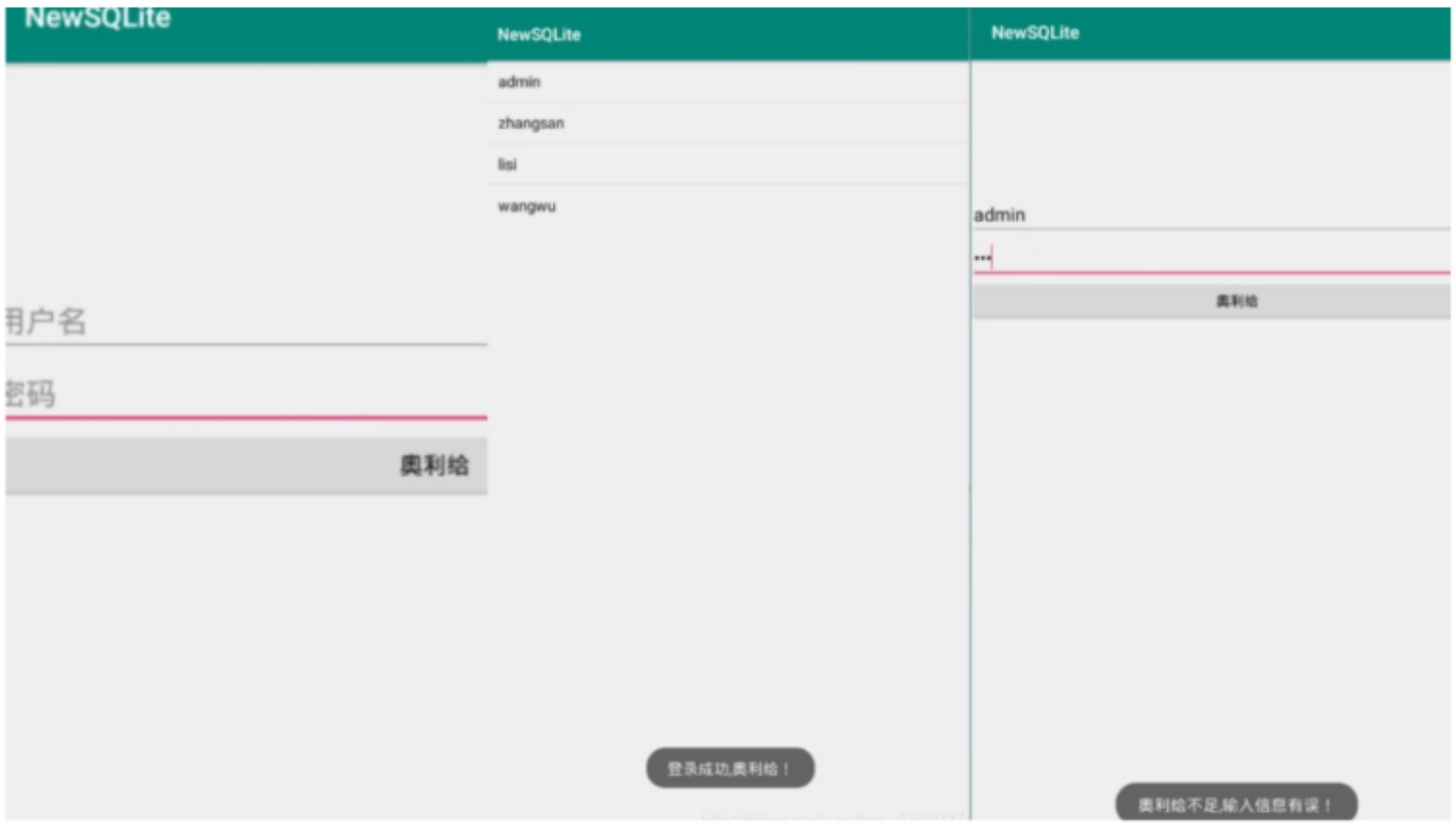
```
public class SQLiteListActivity extends AppCompatActivity {

        ListView listView01;
        SQLiteDBHelper dbHelper;

        @Override
        protected void onCreate(Bundle savedInstanceState) {
                super.onCreate(savedInstanceState);
                setContentView(R.layout.activity_sqlite_list);

                // 提示信息
                Toast.makeText(this,"登录成功,奥利给！ ",Toast.LENGTH_LONG).show();

                listView01 = findViewById(R.id.listView01);

                // 创建 SQLiteDBHelper 对象,利用构造方法进行初始化赋值
                dbHelper = new SQLiteDBHelper(this,"sqlite.db",1);

                // 通过 id 找到 布局管理器中的 相关属性
                SQLiteDatabase db = dbHelper.getReadableDatabase();

                Cursor cursor = db.query("user",new
String[]{"_id","name"},null,null,null,null,null,null);

                // 适配器,利用适配器进行填充信息,相对于数组来说更加灵活,动态化(为这种设计
点赞)
                SimpleCursorAdapter adapter = new SimpleCursorAdapter(
                        this,
                        android.R.layout.simple_list_item_1,
                        cursor,
                        new String[]{"_id","name"},
                        new int[]{android.R.id.text1,android.R.id.text1},
                        0
                );
                listView01.setAdapter(adapter);
        }

        /**
         *  关闭 dbHelper.
         * */
        @Override
        protected void onDestroy() {
                super.onDestroy();
                dbHelper.close();
        }
```

SQLiteLoginActivity 代码：

```
public class SQLiteLoginActivity extends AppCompatActivity {

        SQLiteDBHelper dbHelper;

        EditText etUsername;
        EditText etPassword;
```

```
@Override
protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_sqlite_login);

        // 创建 SQLiteDBHelper 对象,利用构造方法进行初始化赋值
        dbHelper = new SQLiteDBHelper(this,"sqlite.db",1);

        // 通过 id 找到 布局管理器中的 相关属性
        etUsername = findViewById(R.id.etUsername);
        etPassword = findViewById(R.id.etPassword);

}

// 当用户点击提交按钮时,就会到这里(单击事件)
public void onClick(View view) {

        // 获取用户的用户名和密码进行验证
        String username = etUsername.getText().toString();
        String password = etPassword.getText().toString();


        /**
         * Create and/or open a database.
         * */
        SQLiteDatabase db = dbHelper.getReadableDatabase();

        /**
         *
         *   查询用户信息,放回值是一个游标(结果集,遍历结果集)
         *
         * */
        Cursor cursor = db.query("user",new String[]{"name","password"},
"name=?",new String[]{username},null,null,null,"0,1");

        // 游标移动进行校验
        if(cursor.moveToNext()) {
            // 从数据库获取密码进行校验
            @SuppressLint("Range") String dbPassword =
cursor.getString(cursor.getColumnIndex("password"));
            // 关闭游标
            cursor.close();
            if(password.equals(dbPassword)) {
                // 校验成功则跳转到 SQLiteListActivity
                Intent intent = new Intent(this, SQLiteListActivity.class);
                // 启动
                startActivity(intent);
                return;
            }

        }

        // 跳转失败也要进行关闭
        cursor.close();
        // 跳转失败就提示用户相关的错误信息
        Toast.makeText(this," 奥 利 给 不 足 , 输 入 信 息 有 误 !
",Toast.LENGTH_LONG).show();
```

```
    }

    @Override
    public void onDestroy() {
        super.onDestroy();
        dbHelper.close();
    }
}
```



第八章作业：

p271 8.1 使用 Camera2 实现定时拍照的功能。

首先自定义拍照会用到 SurfaceView 控件显示照片的预览区域，以下是布局文件：

```
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
 android:layout_width="match_parent"
 android:layout_height="match_parent" >

 <SurfaceView
  android:id="@+id/surface_view"
  android:layout_width="match_parent"
  android:layout_height="match_parent" />

<RelativeLayout
 android:layout_width="match_parent"
 android:layout_height="match_parent"
 android:orientation="vertical" >

 <ImageView
  android:id="@+id/start"
  android:layout_width="wrap_content"
  android:layout_height="wrap_content"
  android:layout_centerHorizontal="true"
  android:layout_alignParentBottom="true"
```

```
    android:layout_marginBottom="10dp"
    android:src="@drawable/capture"/>

  <TextView
    android:id="@+id/count_down"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:layout_gravity="center"
    android:gravity="center"
    android:textSize="80sp"/>
 </RelativeLayout>
</FrameLayout>
```

MainActivity.java

```
public class MainActivity extends AppCompatActivity implements SurfaceHolder.Callback,
   View.OnClickListener, Camera.PictureCallback {
 private SurfaceView mSurfaceView;
 private ImageView mIvStart;
 private TextView mTvCountDown;

 private SurfaceHolder mHolder;


 private Camera mCamera;

 private Handler mHandler = new Handler();

 private int mCurrentTimer = 10;

 private boolean mIsSurfaceCreated = false;
 private boolean mIsTimerRunning = false;

 private static final int CAMERA_ID = 0; //后置摄像头
// private static final int CAMERA_ID = 1; //前置摄像头
 private static final String TAG = MainActivity.class.getSimpleName();

 @Override
 protected void onCreate(Bundle savedInstanceState) {
  super.onCreate(savedInstanceState);
  setContentView(R.layout.activity_main);

  initView();
  initEvent();
 }

 @Override
 protected void onPause() {
  super.onPause();

  stopPreview();
 }

 private void initView() {
  mSurfaceView = (SurfaceView) findViewById(R.id.surface_view);
  mIvStart = (ImageView) findViewById(R.id.start);
  mTvCountDown = (TextView) findViewById(R.id.count_down);
 }

 private void initEvent() {
```

```
  mHolder = mSurfaceView.getHolder();
  mHolder.addCallback(this);

  mIvStart.setOnClickListener(this);
}

@Override
public void surfaceCreated(SurfaceHolder holder) {
  mIsSurfaceCreated = true;
}

@Override
public void surfaceChanged(SurfaceHolder holder, int format, int width, int height) {
  startPreview();
}

@Override
public void surfaceDestroyed(SurfaceHolder holder) {
  mIsSurfaceCreated = false;
}

private void startPreview() {
  if (mCamera != null || !mIsSurfaceCreated) {
    Log.d(TAG, "startPreview will return");
    return;
  }

  mCamera = Camera.open(CAMERA_ID);

  Camera.Parameters parameters = mCamera.getParameters();
  int width = getResources().getDisplayMetrics().widthPixels;
  int height = getResources().getDisplayMetrics().heightPixels;
  Camera.Size size = getBestPreviewSize(width, height, parameters);
  if (size != null) {
    //设置预览分辨率
    parameters.setPreviewSize(size.width, size.height);
    //设置保存图片的大小
    parameters.setPictureSize(size.width, size.height);
  }

  //自动对焦
  parameters.setFocusMode(Camera.Parameters.FOCUS_MODE_AUTO);
  parameters.setPreviewFrameRate(20);

  //设置相机预览方向
  mCamera.setDisplayOrientation(90);

  mCamera.setParameters(parameters);

  try {
    mCamera.setPreviewDisplay(mHolder);
  } catch (Exception e) {
    Log.d(TAG, e.getMessage());
  }

  mCamera.startPreview();
}
```

```
private void stopPreview() {
  //释放 Camera 对象
  if (mCamera != null) {
    try {
      mCamera.setPreviewDisplay(null);
      mCamera.stopPreview();
      mCamera.release();
      mCamera = null;
    } catch (Exception e) {
      Log.e(TAG, e.getMessage());
    }
  }
}

private Camera.Size getBestPreviewSize(int width, int height,
                Camera.Parameters parameters) {
  Camera.Size result = null;

  for (Camera.Size size : parameters.getSupportedPreviewSizes()) {
    if (size.width <= width && size.height <= height) {
      if (result == null) {
        result = size;
      } else {
        int resultArea = result.width * result.height;
        int newArea = size.width * size.height;

        if (newArea > resultArea) {
          result = size;
        }
      }
    }
  }

  return result;
}

@Override
public void onClick(View v) {
  switch (v.getId()) {
    case R.id.start:
      if (!mIsTimerRunning) {
        mIsTimerRunning = true;
        mHandler.post(timerRunnable);
      }
      break;
  }
}

private Runnable timerRunnable = new Runnable() {
  @Override
  public void run() {
    if (mCurrentTimer > 0) {
      mTvCountDown.setText(mCurrentTimer + "");

      mCurrentTimer--;
      mHandler.postDelayed(timerRunnable, 1000);
    } else {
      mTvCountDown.setText("");
```

```
      mCamera.takePicture(null, null, null, MainActivity.this);
      playSound();

      mIsTimerRunning = false;
      mCurrentTimer = 10;
    }
  }
};

@Override
public void onPictureTaken(byte[] data, Camera camera) {
  try {
    FileOutputStream fos = new FileOutputStream(new File
        (Environment.getExternalStorageDirectory() + File.separator +
            System.currentTimeMillis() + ".png"));

    //旋转角度，保证保存的图片方向是对的
    Bitmap bitmap = BitmapFactory.decodeByteArray(data, 0, data.length);
    Matrix matrix = new Matrix();
    matrix.setRotate(90);
    bitmap = Bitmap.createBitmap(bitmap, 0, 0,
        bitmap.getWidth(), bitmap.getHeight(), matrix, true);
    bitmap.compress(Bitmap.CompressFormat.PNG, 100, fos);
    fos.flush();
    fos.close();
  } catch (FileNotFoundException e) {
    e.printStackTrace();
  } catch (IOException e) {
    e.printStackTrace();
  }

  mCamera.startPreview();
}

/**
 * 播放系统拍照声音
 */
public void playSound() {
  MediaPlayer mediaPlayer = null;
  AudioManager audioManager = (AudioManager)
getSystemService(Context.AUDIO_SERVICE);
  int volume = audioManager.getStreamVolume( AudioManager.STREAM_NOTIFICATION);

  if (volume != 0) {
    if (mediaPlayer == null)
      mediaPlayer = MediaPlayer.create(this,
          Uri.parse("file:///system/media/audio/ui/camera_click.ogg"));
    if (mediaPlayer != null) {
      mediaPlayer.start();
    }
  }
 }
}
```

有两点需要注意：对于 Camera 来说，默认是横屏的，所以预览的时候和图片保存的时候都是横屏的，需要调整角度。

设置相机预览方法：

```
//设置相机预览方向
mCamera.setDisplayOrientation(90);
```

保存图片的时候调整角度：

```
@Override
public void onPictureTaken(byte[] data, Camera camera) {
    try {
        FileOutputStream fos = new FileOutputStream(new File
                (Environment.getExternalStorageDirectory() + File.separator + System.currentTimeMillis() + "
        //旋转角度，保证保存的图片方向是对的
        Bitmap bitmap = BitmapFactory.decodeByteArray(data, 0, data.length);
        Matrix matrix = new Matrix();
        matrix.setRotate(90);
        bitmap = Bitmap.createBitmap(bitmap, 0, 0, bitmap.getWidth(), bitmap.getHeight(), matrix, true);
        bitmap.compress(Bitmap.CompressFormat.PNG, 100, fos);
        fos.flush();
        fos.close();
    } catch (FileNotFoundException e) {
        e.printStackTrace();
    } catch (IOException e) {
        e.printStackTrace();
    }

    mCamera.startPreview();
}
```

```
        if (mediaPlayer == null)
            mediaPlayer = MediaPlayer.create(this,
                Uri.parse("file:///system/media/audio/ui/camera_click.ogg"));
        if (mediaPlayer != null) {
            mediaPlayer.start();
        }
    }
}
```

有两点需要注意，对于 Camera 来说，默认是横屏的，所以预览的时候和图片保存的时候都是横屏的，需要调整角度。

设置相机预览方法，

```
//设置相机预览方向
mCamera.setDisplayOrientation(90);
```

保存图片的时候调整角度，

```
@Override
public void onPictureTaken(byte[] data, Camera camera) {
    try {
        FileOutputStream fos = new FileOutputStream(new File
                (Environment.getExternalStorageDirectory() + File.separator + System.currentTimeMillis() + ".jpg"));
        //旋转角度，保证保存的图片方向是对的
        Bitmap bitmap = BitmapFactory.decodeByteArray(data, 0, data.length);
        Matrix matrix = new Matrix();
        matrix.setRotate(90);
        bitmap = Bitmap.createBitmap(bitmap, 0, 0, bitmap.getWidth(), bitmap.getHeight(), matrix, true);
        bitmap.compress(Bitmap.CompressFormat.PNG, 100, fos);
        fos.flush();
        fos.close();
    } catch (FileNotFoundException e) {
```